

Windows Migration Tool

User Manual

May 2026

A complete guide to exporting and restoring your Windows environment.

Contents

1	Introduction	3
2	System Requirements	3
3	Installation	3
3.1	Option A: Download from GitHub Releases	3
3.2	Option B: Build the Installer Yourself	3
3.3	Option C: Portable (No Install, from Source)	4
4	Usage	4
4.1	GUI Mode	4
4.2	Command-Line Mode	4
4.2.1	Exporting (On the Old Machine)	4
4.2.2	Restoring (On the New Machine)	4
5	Migration Workflow	5
6	Output Structure	5
6.1	File Descriptions	5
7	What the Restore Script Does	6
8	Post-Restore Steps	6
9	Limitations	6
10	Troubleshooting	7
11	Continuous Integration & Releases	7
12	Project Files Reference	7

1 Introduction

The Windows Migration Tool is a PowerShell-based application (with an optional GUI) that helps you move your environment from one Windows 10 or 11 machine to another. It automates the tedious parts of PC migration:

- Exporting WSL distributions as `.tar` archives
- Inventorying all installed software (Win32 and Store apps)
- Retrieving license keys (Windows, Office, registry-stored serials)
- Exporting winget packages for bulk reinstall
- Generating a self-contained restore script

2 System Requirements

- Windows 10 (version 1709 or later) or Windows 11 on the **source** machine
- Windows 11 on the **destination** machine
- PowerShell 5.1 or later
- **Administrator privileges** (the tool must be run as Admin)
- An external drive or network share with sufficient space for WSL exports
- winget (pre-installed on Windows 11; on Windows 10, install from the [Microsoft Store](#) or [GitHub](#))

3 Installation

3.1 Option A: Download from GitHub Releases

1. Go to the [Releases](#) page on GitHub.
2. Download `MigrationTool.Setup.exe` (installer) or `MigrationTool.Portable.zip` (no install required).
3. For the installer: run it—it creates a Start Menu shortcut and an optional desktop icon.
4. For portable: extract the zip and run `MigrationTool.cmd`.

Note: Windows SmartScreen may block the unsigned installer. Click “**More info**” then “**Run anyway**” to proceed.

3.2 Option B: Build the Installer Yourself

1. Download and install [Inno Setup 6](#).
2. Compile `installer.iss` to produce `MigrationTool.Setup.exe`.
3. Run the installer—it creates a Start Menu shortcut and an optional desktop icon.

3.3 Option C: Portable (No Install, from Source)

Simply double-click `MigrationTool.cmd` or run `MigrationTool-GUI.ps1` directly from an elevated PowerShell prompt. No installation is required.

4 Usage

4.1 GUI Mode

Launch the application from the Start Menu, the desktop shortcut, or by running `MigrationTool.cmd`.

1. Click **Browse** to select your external drive or output folder.
2. Click **Export** to back up this machine (export only).
3. Click the ▾ arrow next to Export for options: “Export Only” or “Export + Create Restore Bundle”.
4. On the destination machine, click **Browse** to locate the export folder, then click **Restore**.
5. Click **Cancel** at any time to stop a running operation.

The GUI features:

- Dark (Catppuccin-style) color scheme
- Prominent path label with helper text
- Live log output showing progress
- Progress bar
- Cancel button to abort operations
- Automatic self-elevation to Administrator
- Non-blocking UI (export/restore runs in background threads)

4.2 Command-Line Mode

4.2.1 Exporting (On the Old Machine)

```
# Export to an external drive
.\Migrate-Machine.ps1 -OutputPath "E:\MigrationExport"

# Export and create a self-contained restore zip
.\Migrate-Machine.ps1 -OutputPath "E:\MigrationExport" -Bundle

# Or use the default (Desktop\MigrationExport)
.\Migrate-Machine.ps1
```

4.2.2 Restoring (On the New Machine)

Move the external drive to the new machine, then run:

```
E:\MigrationExport\Restore-Machine.ps1
```

Or specify the import path manually:

```
.\Restore-Machine.ps1 -ImportPath "E:\MigrationExport"
```

5 Migration Workflow

The recommended end-to-end workflow:

1. Plug an external drive into the **old machine**.
2. Run the export (GUI or command line) targeting the external drive.
3. Wait for the export to complete (WSL archives may take time for large distros).
4. Move the drive to the **new machine**.
5. Run the restore script from the export folder.
6. Complete the post-restore steps (see Section 8).

6 Output Structure

The export produces a self-contained folder with the following layout:

```
MigrationExport/
  license_keys.txt          # Windows/Office/software keys
  installed_software.csv    # Full inventory (machine-readable)
  installed_software.txt    # Full inventory (human-readable)
  installed_software.html   # Interactive checklist with download links
  winget_packages.json     # For 'winget import'
  WSL/
    Ubuntu.tar              # (example) WSL distro archive
    .wslconfig              # WSL global config
  Restore-Machine.ps1      # Run this on the new machine
  migration_log_*.txt      # Export log
```

6.1 File Descriptions

license_keys.txt

Contains product keys retrieved from BIOS/UEFI, Office `ospp.vbs`, and a registry scan for common key names (Serial, LicenseKey, ProductKey, CDKey, etc.).

installed_software.csv

Machine-readable inventory of all installed applications with names, versions, and download URLs.

installed_software.txt

Human-readable version of the software inventory.

installed_software.html

Interactive checklist with checkboxes, download/search links, and license keys (hidden by default with a copy button). Open in any browser on the new machine.

winget_packages.json

Package list for `winget import` to automate bulk reinstallation.

WSL/*.tar

Each WSL distribution exported as a tar archive.

WSL/.wslconfig

Global WSL configuration file.

Restore-Machine.ps1

Auto-generated restore script that uses `$PSScriptRoot` to locate sibling data files.

migration_log_*.txt

Timestamped log of the export process.

7 What the Restore Script Does

The generated `Restore-Machine.ps1` script performs:

1. **winget import** — Reads `winget_packages.json` and installs all listed packages.
2. **WSL import** — Runs `wsl --import` for each `.tar` archive in the `WSL/` folder, restoring your Linux distributions.

The script is self-contained: it uses `$PSScriptRoot` so it works from wherever the export folder is located on the new machine.

8 Post-Restore Steps

After running the restore script, complete these manual steps:

1. **Set WSL default users:** For each restored distro, run:
`<distro> config --default-user <username>`
2. **Install remaining apps:** Check `installed_software.csv` for applications not available via winget and install them manually.
3. **Re-enter license keys:** Open `license_keys.txt` and activate software that requires manual key entry.
4. **Restore app-specific settings:** Any application configurations not covered by this tool (browser profiles, IDE settings, etc.) must be transferred separately.

9 Limitations

- License keys are only retrievable if stored in BIOS/UEFI or the Windows registry. Digital licenses tied to a Microsoft account transfer automatically and will not appear in the export.
- Some winget packages may fail to import if the source or package ID has changed since the export.
- WSL exports include the full filesystem—large distributions will produce large `.tar` files. Ensure your external drive has sufficient free space.
- Framework and runtime packages from the Microsoft Store are filtered out to reduce noise, but some may still appear.

10 Troubleshooting

“Access Denied” or permission errors

Ensure you are running the tool as Administrator. The GUI self-elevates automatically; for command-line use, right-click PowerShell and select “Run as administrator.”

WSL export fails

Verify that WSL is installed and that your distributions are in a stopped state. Run `wsl --shutdown` before exporting.

winget import partially fails

Some packages may have been removed or renamed. Check the log file and install those packages manually.

Export folder is very large

WSL distributions contain full Linux filesystems. Consider cleaning up unnecessary files inside WSL before exporting (e.g., package caches, temp files).

11 Continuous Integration & Releases

The project includes GitHub Actions workflows for automated builds:

`.github/workflows/build-installer.yml`

Runs on every push and pull request to `main`. Compiles the Inno Setup installer and uploads it as a workflow artifact.

`.github/workflows/release.yml`

Triggered when a version tag (e.g., `v1.0.0`) is pushed. Builds the installer, packages a portable zip, and publishes both as GitHub Release assets.

To create a release:

```
git tag v1.0.0
git push origin v1.0.0
```

This produces a GitHub Release with two downloadable assets:

- `MigrationTool_Setup.exe` — full Windows installer
- `MigrationTool_Portable.zip` — portable scripts (no installation required)

12 Project Files Reference

`Migrate-Machine.ps1`

Main export script. Accepts `-OutputPath` parameter.

`MigrationTool-GUI.ps1`

WPF-based graphical interface.

`MigrationTool.cmd`

CMD launcher that starts the GUI without a console flash.

`installer.iss`

Inno Setup 6 script for building the Windows installer.

`icon.ico`

Application icon used by the installer.

`.github/workflows/build-installer.yml`

CI workflow for building the installer on push/PR.

`.github/workflows/release.yml`

Release workflow for publishing tagged versions.